1. Probabilistic Graphical Models

The network is $S \to L$, $T \to E \leftarrow L$, $E \to X$. The evidence $X = 1$ is applied first: the factor $P(X \mid E)$ is restricted to $X = 1$, which gives

$$p(E) = P(X = 1 \mid E): \qquad p(E{=}1) = 0.9, \quad p(E{=}0) = 0.1.$$

The factors are then $P(S)$, $P(L \mid S)$, $P(T)$, $P(E \mid T, L)$ and $p(E)$. The query variable is L, so the variables to eliminate are S, T, E, in this order.

Step 1: eliminate S.

(1) Factors multiplied: $P(S)$ and $P(L \mid S)$.
(2) Scope of the product: $\{S, L\}$.
(3) New factor $f_1(L) = \sum_S P(S)\, P(L \mid S)$:

L	$f_1(L)$
0	$0.5 \cdot 0.95 + 0.5 \cdot 0.80 = 0.875$
1	$0.5 \cdot 0.05 + 0.5 \cdot 0.20 = 0.125$

Step 2: eliminate T.

(1) Factors multiplied: $P(T)$ and $P(E \mid T, L)$.
(2) Scope of the product: $\{T, E, L\}$.
(3) New factor $f_2(E, L) = \sum_T P(T)\, P(E \mid T, L)$:

E	L	$f_2(E, L)$
1	1	$0.9 \cdot 1 + 0.1 \cdot 1 = 1$
0	1	$0.9 \cdot 0 + 0.1 \cdot 0 = 0$
1	0	$0.9 \cdot 0 + 0.1 \cdot 1 = 0.1$
0	0	$0.9 \cdot 1 + 0.1 \cdot 0 = 0.9$

Step 3: eliminate E.

(1) Factors multiplied: $f_2(E, L)$ and $p(E)$.
(2) Scope of the product: $\{E, L\}$.
(3) New factor $f_3(L) = \sum_E f_2(E, L)\, p(E)$:

L	$f_3(L)$
0	$0.1 \cdot 0.9 + 0.9 \cdot 0.1 = 0.18$
1	$1 \cdot 0.9 + 0 \cdot 0.1 = 0.9$

Final step: combine and normalise. The remaining factors $f_1(L)$ and $f_3(L)$ are multiplied:

$$f_1(0) f_3(0) = 0.875 \cdot 0.18 = 0.1575, \qquad f_1(1) f_3(1) = 0.125 \cdot 0.9 = 0.1125.$$

Their sum is $0.27 = P(X = 1)$. Normalising gives

$$P(L = 1 \mid X = 1) = \frac{0.1125}{0.27} = \frac{5}{12} \approx 0.417, \qquad P(L = 0 \mid X = 1) = \frac{7}{12} \approx 0.583.$$

2. Hidden Markov Models

The hidden states are R and S, and the observed sequence is $X_1 = y$, $X_2 = y$, $X_3 = n$. Filtering requires only the forward pass. Let $\alpha_t(z) = P(Z_t = z, x_{1:t})$. Then

$$\alpha_1(z) = P(Z_1 = z)\, P(x_1 \mid z), \qquad \alpha_t(z) = P(x_t \mid z) \sum_{z'} \alpha_{t-1}(z')\, P(Z_t = z \mid Z_{t-1} = z'),$$

and the filtered distribution is $P(Z_t = z \mid x_{1:t}) = \alpha_t(z) / \sum_{z'} \alpha_t(z')$. The emission probabilities are $P(y \mid R) = 0.9$, $P(y \mid S) = 0.2$, $P(n \mid R) = 0.1$ and $P(n \mid S) = 0.8$.

$t = 1$, $x_1 = y$.

$$\alpha_1(R) = 0.5 \cdot 0.9 = 0.45, \qquad \alpha_1(S) = 0.5 \cdot 0.2 = 0.10.$$

The sum is 0.55, so

$$P(Z_1 \mid x_1) = \big(R : 0.8182,\ S : 0.1818\big).$$

$t = 2$, $x_2 = y$. Prediction step:

$$\sum_{z'} \alpha_1(z') P(R \mid z') = 0.45{\cdot}0.7 + 0.10{\cdot}0.4 = 0.355, \qquad \sum_{z'} \alpha_1(z') P(S \mid z') = 0.45{\cdot}0.3 + 0.10{\cdot}0.6 = 0.195.$$

Update step:

$$\alpha_2(R) = 0.9 \cdot 0.355 = 0.3195, \qquad \alpha_2(S) = 0.2 \cdot 0.195 = 0.0390.$$

The sum is 0.3585, so

$$P(Z_2 \mid x_{1:2}) = \big(R : 0.8912,\ S : 0.1088\big).$$

$t = 3$, $x_3 = n$. Prediction step:

$$0.3195 \cdot 0.7 + 0.0390 \cdot 0.4 = 0.23925, \qquad 0.3195 \cdot 0.3 + 0.0390 \cdot 0.6 = 0.11925.$$

Update step:

$$\alpha_3(R) = 0.1 \cdot 0.23925 = 0.023925, \qquad \alpha_3(S) = 0.8 \cdot 0.11925 = 0.0954.$$

The sum is $0.119325 = P(x_{1:3})$, so

$$P(Z_3 \mid x_{1:3}) = \big(R : 0.2005,\ S : 0.7995\big).$$

4. Gaussian Processes

The squared exponential kernel $k(x, x') = \exp\big(-\|x - x'\|^2/(2l^2)\big)$. The `rbfdot` kernel in `kernlab` is $\exp(-\sigma\|x - x'\|^2)$, so $\sigma = 1/(2l^2) = \frac{1}{2}\exp(-2K)$ with $l = \exp(K)$. The parameter K is unconstrained, which guarantees $l > 0$.

```
############################################################
## 1. Loss calculation
############################################################
loss_fn <- function(K, Xtr, ytr) {
  l     <- exp(K)
  sigma <- 1 / (2 * l^2)

  # Fit the Gaussian process classifier with this bandwidth
  gp <- gausspr(Xtr, ytr, type = "classification",
                kernel = "rbfdot", kpar = list(sigma = sigma),
                scaled = FALSE)

  # Predicted probability of class 1 (second column = level "1")
  p <- predict(gp, Xtr, type = "probabilities")[, 2]
  p <- pmin(pmax(p, 1e-12), 1 - 1e-12)   # avoid log(0)

  # L = -(1/n) * sum( y*log(p) + (1-y)*log(1-p) )
  -mean(ytr * log(p) + (1 - ytr) * log(1 - p))
}

############################################################
## 2. Gradient calculation
############################################################
grad_fn <- function(K, Xtr, ytr, eps = 1e-3) {
  (loss_fn(K + eps, Xtr, ytr) -
   loss_fn(K - eps, Xtr, ytr)) / (2 * eps)
}

############################################################
## 3. Parameter updating (one gradient descent step)
############################################################
update_fn <- function(K, grad, lr) {
  K - lr * grad
}

############################################################
## Gradient descent loop
############################################################
K       <- 0      # initial value: l = exp(0) = 1
lr      <- 0.5    # learning rate
n_iter  <- 30     # number of iterations

for (i in 1:n_iter) {
  g <- grad_fn(K, Xtr, ytr)   # gradient at the current K
  K <- update_fn(K, g, lr)              # gradient descent step
}
```